RICE UNIVERSITY

Diagnosing Distributed Systems

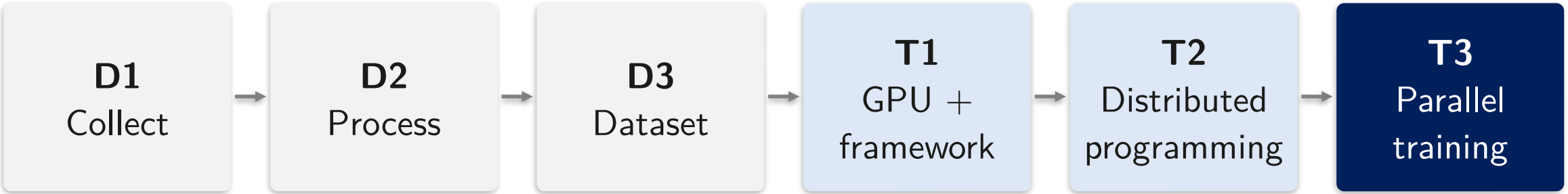
Reading timelines, finding stragglers, counting useful FLOPs and joules, and surviving failures

Yuke Wang

Rice University

COMP 329 · Spring 2027 · Week 13, Lecture 2

Where we are in the pipeline



Lecture 1: pipeline parallelism, the bubble, and d $\times$ t $\times$ p layouts

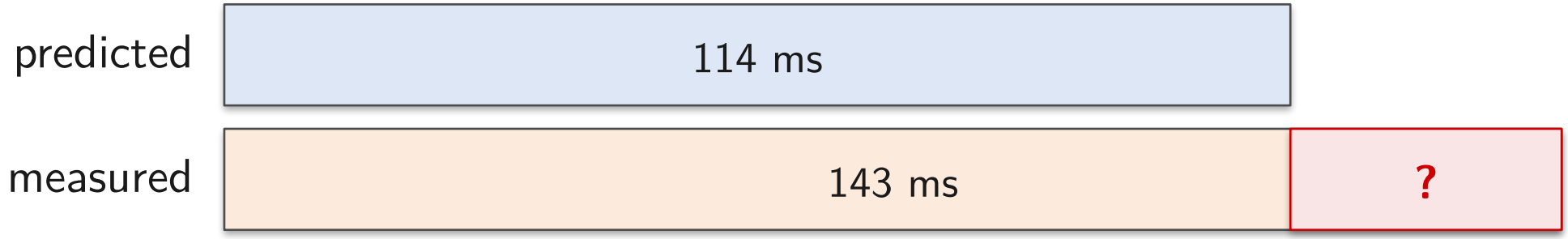
In our pipeline

Today: when the real run is slower than the model, **find out why** — and keep the run alive.

This week: **capstone pairs** are set. Week 14: integration and review; **T3 due**.

Why do we need diagnosis tools?

Your cost model predicts **114 ms** per step on 8 GPUs. Every GPU reports **143 ms**. No error; the loss falls normally.

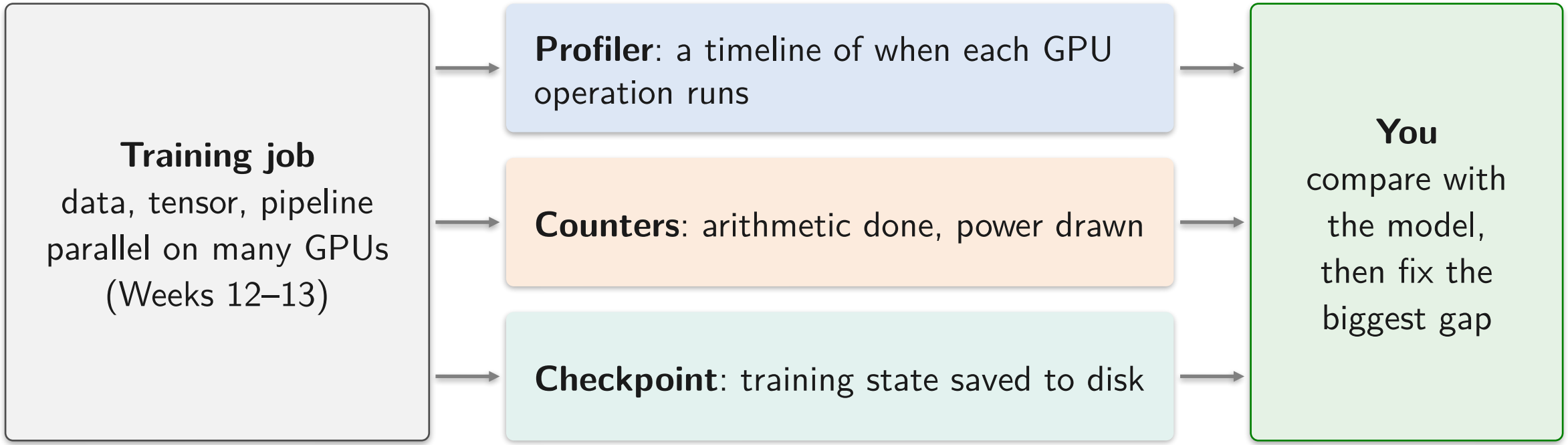


Where did the extra time go, and how would you find out?

Answer

20% of every GPU's time is lost, and the total cannot say why: network? one slow GPU? data loading? We must **look inside** the step.

Diagnosis tools, in plain words



Key idea

Diagnosis: **measure** the real run, compare it with the **model**, fix the **biggest** gap.

Where diagnosis shows up

- **Timelines, counters**: PyTorch profiler, Nsight Systems, nvidia-smi.
- **Checkpoints**: PyTorch Distributed Checkpoint saves each GPU's share.
- **At scale**: Llama 3 (16,384 GPUs) built tools to find slow and failed GPUs.
- **Cost**: profiling slows a run; checkpoints take time to write.
- **In this course**: the scaling section of your T3 report; the capstone.

In our pipeline

Why diagnosis tools? At scale a step runs at the pace of the **slowest** GPU and a job dies with the **first** failure: measuring is how we find both.

Source: Llama Team, Meta, "The Llama 3 Herd of Models," 2024, §3.3.4. PyTorch documentation (torch.profiler, DCP).

Learning goals for today

- **Read** a profiler timeline: compute vs communication, **exposed** vs overlapped.
- **Explain** why the **slowest GPU** sets the step time, and find it.
- **Measure** useful work (MFU) and **energy**, and say why "GPU utilization %" misleads.
- **State** what an exact, safe **restart** needs, and how to run a fair scaling test.

A puzzle to start

nvidia-smi (NVIDIA's monitoring tool)
GPU-Util: **100%**

A training job shows 100% GPU utilization. Is the GPU doing as much arithmetic as it can?

Answer

No. It only means *some* kernel was running all the time. One busy processor (SM) out of 132 already counts as 100%.

Source: NVIDIA System Management Interface (nvidia-smi) documentation; NVML API reference.

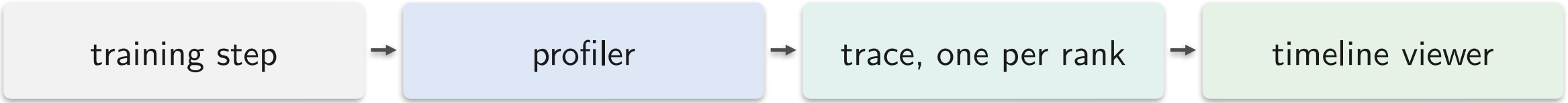
Part 1

Reading a profiler timeline

Where did the time in one training step go?

What a profiler records

- **kernel**: one function the GPU runs (a matrix product, an all-reduce).
- **stream**: a queue of kernels run in order; two streams can run **at the same time**.
- **profiler**: records each kernel's stream, start, and end — one trace **per rank**.

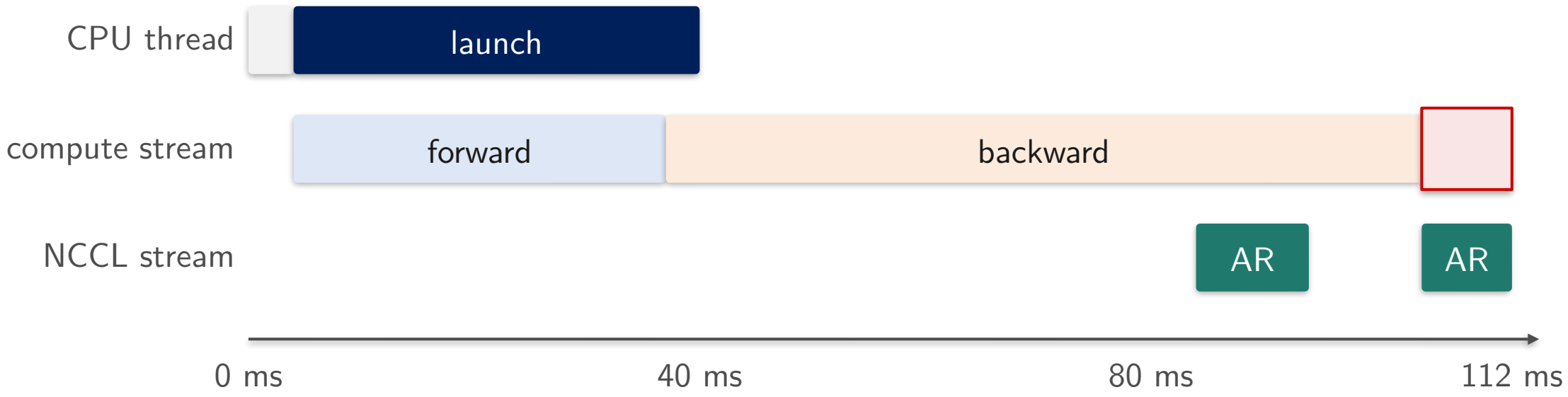


Key idea

Profile **every** rank: a slow rank shows up as **waiting** on all the others.

Source: PyTorch documentation, torch.profiler; NVIDIA Nsight Systems User Guide.

Anatomy of a timeline



exposed: compute idle, NCCL busy

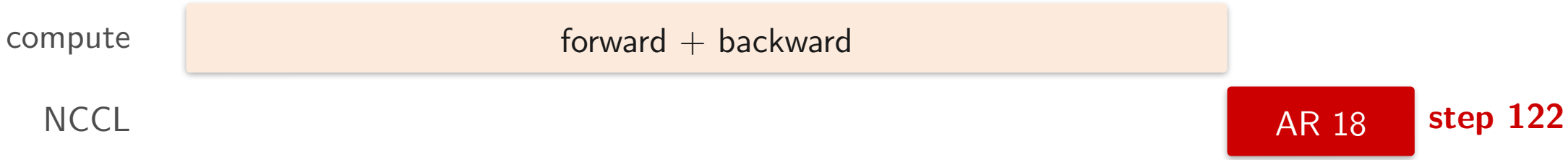
Key idea

Read the **compute** row first: every gap there is time you pay for.

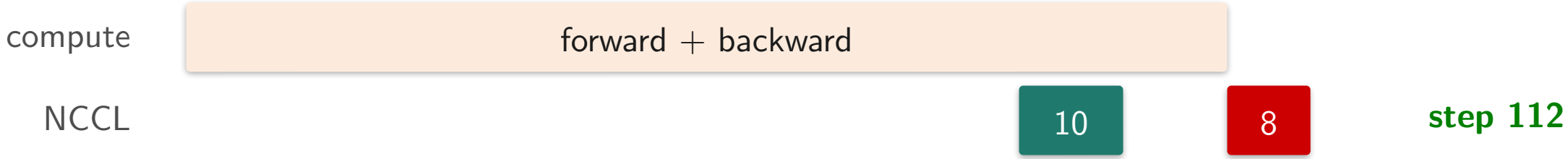
Source: PyTorch DDP documentation (gradient buckets).

Overlapped versus exposed communication

Reduce after backward



Bucketed (DDP): overlap with backward



Key idea

Overlapped communication is hidden under compute; only **exposed** communication lengthens the step.

Accounting for a step

$$T(\text{step}) = T(\text{compute}) + T(\text{exposed comm}) + T(\text{other idle})$$

$$\text{exposed} = \text{communication} - \text{overlapped}$$

$$112\ \text{ms} = 100 + (18 - 10) + 4$$

Watch out

A faster network can save **at most the exposed** 8 ms, not the full 18 ms.

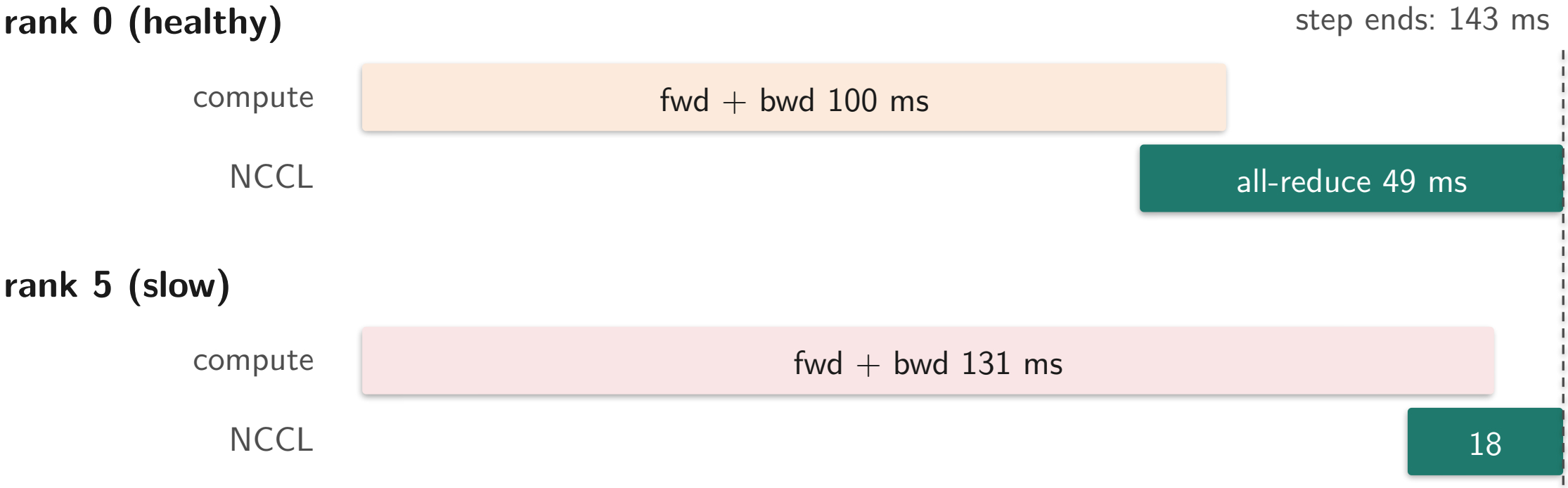
Idle gaps and what causes them

Gap in compute	Other rows show	Typical fix
Before forward	CPU loading data	more loader workers
After printing a GPU value	CPU waiting on the GPU	log less often
After backward	a long NCCL kernel	overlap, or find the late rank

Key idea

Every gap has a cause visible in **another row** of the same timeline.

A wait looks like communication



Watch out

Same collective: 49 ms on rank 0, 18 ms on rank 5. Rank 0 is **waiting**.

Check yourself

Two minutes with a neighbour.

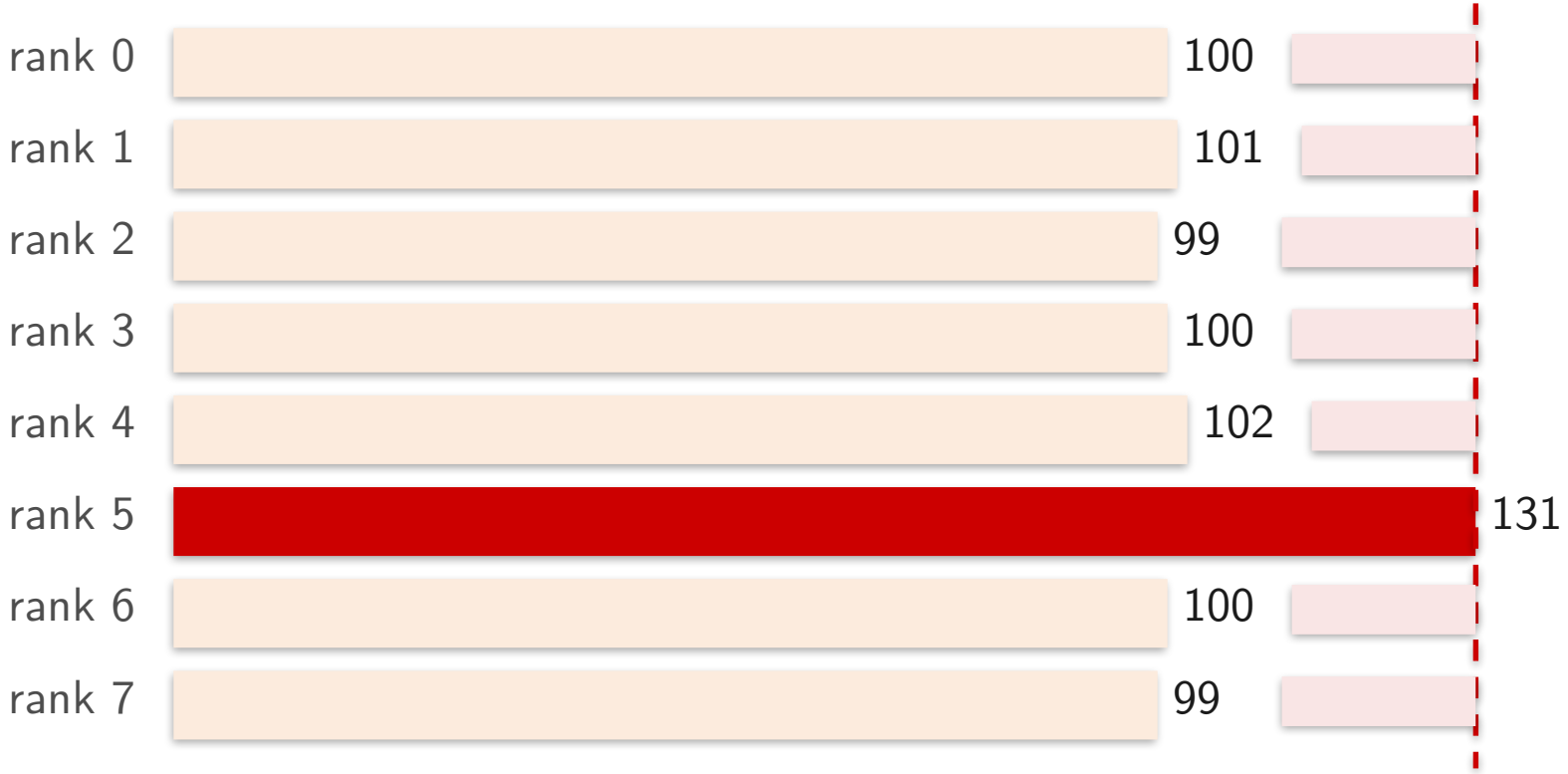
- Does every millisecond of NCCL kernel time add to the step time? Why not?
- Data 4 ms, compute 100 ms, all-reduce 30 ms with 10 ms overlapped: step time?
- Only rank 0's profile shows a 49 ms all-reduce. What do you check next?

Part 2

Stragglers, MFU, and energy

Why does one slow GPU out of a thousand matter?

The slowest rank sets the step



$$T(\text{step}) = \max \text{ over ranks } k \text{ of } T(k) + T(\text{comm})$$

Watch out

Seven ranks wait $\approx$ 30 ms each: **216 ms** of GPU time lost per step to one **straggler**.

A rare hiccup becomes every step

$$P(\text{some rank slow}) \approx 1 - \exp(-q \cdot n)$$

q = chance that one rank has a slow step; n = number of ranks

q = 1%, n = 8
slow step: **7.7%**

q = 1%, n = 512
slow step: **99.4%**

Key idea

A **rare** event per GPU becomes an **every-step** event at scale.

Noise alone: the slowest of n

$$T(k) = 100 + 5 \cdot Z(k) \text{ ms}, \quad Z(k) \text{ standard normal}$$

$$E[\text{ max of } n \text{ standard normals }] \leq \text{ sqrt}(2 \ln n)$$

The slowest of n grows **slowly** — but **without limit**.

Result

n = 1024 ranks, no broken GPU: the step is ≈ **116 ms** instead of 100. Shrinking the **spread** helps as much as shrinking the mean.

Finding the straggler

- The **whole step** is equal on every rank: it cannot point at anyone.
- Compare the **same all-reduce** across ranks: the **shortest** one is the straggler's.
- Or time each rank's **local** compute and compare it with the **median**.

Key idea

Then ask: is it **always the same rank**? Then it is the machine, not the code.

Common causes

Cause	Evidence	Fix
Throttled GPU (hot, power-capped)	lower power and clock	replace or cool
Uneven work	time tracks input size	balance the work
Host noise (logging, other programs)	occasional; often rank 0	log less

Key idea

A **persistent** straggler is hardware or setup; an **occasional** one is noise.

What “GPU utilization %” measures

nvidia-smi definition (paraphrased)

Percent of time during which **at least one kernel** was running on the GPU.

- One kernel on **1 of 132 SMs** the whole time → 100%.
- An all-reduce **waiting** for a slow rank → 100%.

Watch out

Low utilization proves the GPU is idle; **high** utilization proves almost nothing.

Source: NVIDIA System Management Interface (nvidia-smi) documentation; NVML API reference.

Model FLOPs utilization (MFU)

FLOP: one floating-point add or multiply. **Peak**: FLOP/s from the datasheet.

$$\text{MFU} = \text{needed FLOPs} \;/\; (\text{step time} \times \text{GPUs} \times \text{peak})$$

needed FLOPs: the arithmetic the model's formulas require for one step

Key idea

MFU counts only the **arithmetic the model needs**. Waiting, idling, and overhead all lower it.

Source: Chowdhery et al., "PaLM," 2022 (model FLOPs utilization); H100 datasheet (FP32 peak).

MFU for our MLP

$$\text{FLOPs per step} \approx 6 \times \text{weights} \times \text{batch}$$

$$= 6 \times 28{,}800 \times 64 \approx 11.1 \text{ million}$$

$$\text{MFU at 100 microseconds per step} \approx 0.17\%$$

Watch out

Our MLP is tiny: **launch overhead**, not arithmetic, fills the step — even when GPU-Util reads 100%.

Source: NVIDIA H100 Tensor Core GPU datasheet (SXM5); NVIDIA H100 Tensor Core GPU Architecture whitepaper (2022). Step time from Week 8.

Power and energy

power (watts) = joules per second; **energy** (joules) = power × time

$$\text{energy per step} = \Sigma \text{ over GPUs of power} \times \text{step time}$$

$$8 \text{ GPUs} \times 650 \text{ watts} \times 0.112 \text{ seconds} \approx 582 \text{ joules}$$

Key idea

A **waiting** GPU still draws power: a straggler wastes **energy** as well as time.

Source: NVIDIA H100 Tensor Core GPU datasheet (SXM5); NVIDIA H100 Tensor Core GPU Architecture whitepaper (2022). 650 W is this scenario's assumption.

Check yourself

Two minutes with a neighbour.

- Each of 256 ranks has a 1% chance of a slow step. How often is the job's step slow?
- Why is the whole-step time useless for finding the straggler?
- Every GPU shows 100% utilization, yet MFU is 5%. Give two possible reasons.

Part 3

Checkpoints and fair experiments

How do we restart a job exactly where it stopped?

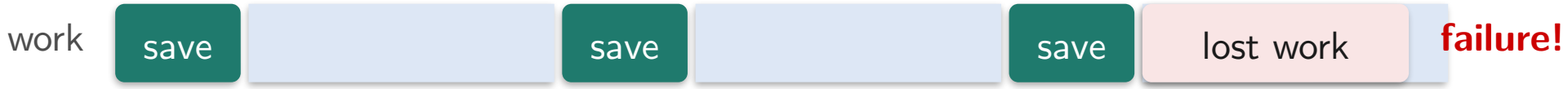
Failures are routine at scale

$$\text{MTBF(job)} = \text{MTBF(one GPU)} \;/\; n$$

MTBF: mean time between failures

Llama 3 training

16,384 GPUs: one unexpected interruption every ≈ **3 hours**.



Key idea

On failure: **all** ranks restart from the last **complete** checkpoint and redo the lost work.

Source: Llama Team, Meta, "The Llama 3 Herd of Models," 2024, §3.3.4.

What an exact restart needs

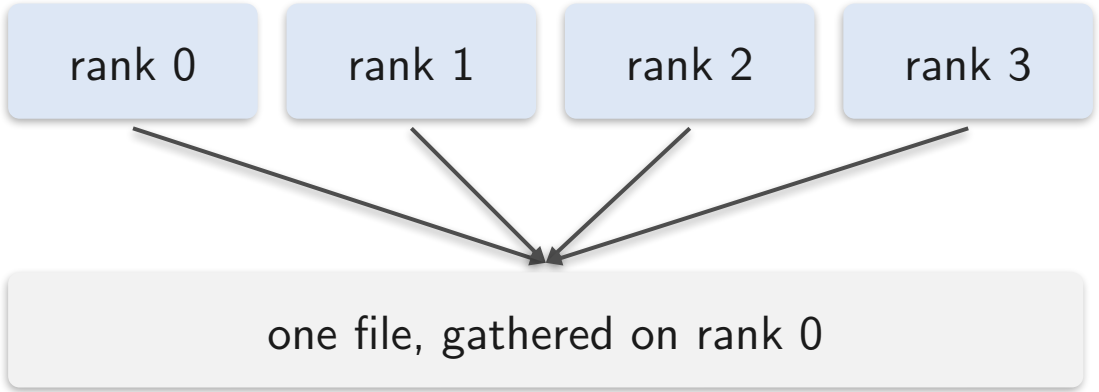
Save	Forget it and ...
Model, optimizer, learning-rate schedule	updates jump; warm-up repeats
Random-number generator states, every rank	different dropout and shuffles
Data position (epoch, batch)	data repeated or skipped

Check

Test it: train 2k steps; separately train k, stop, restart, train k — the losses must **match**.

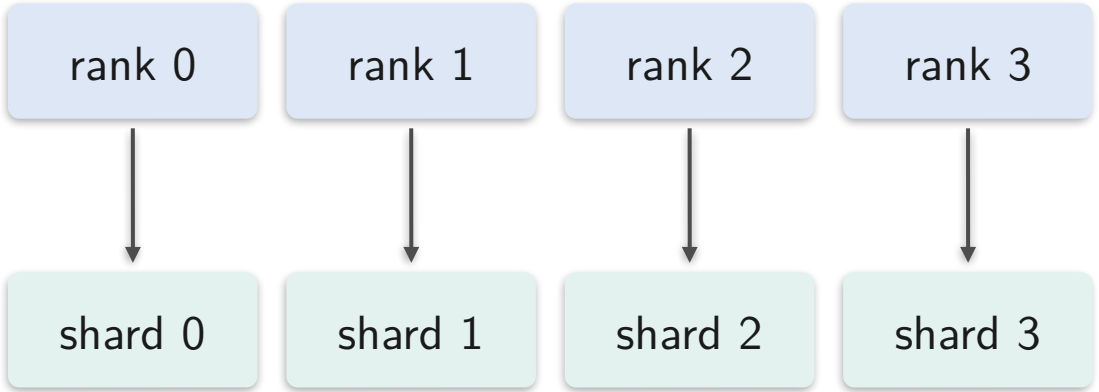
Sharded versus consolidated checkpoints

Consolidated



all state on rank 0; one slow writer

Sharded



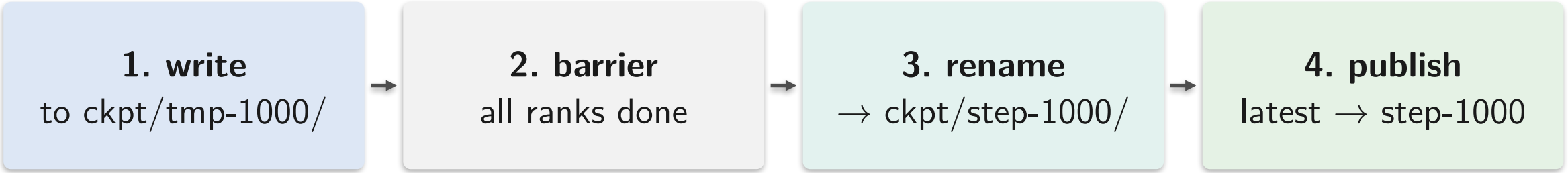
parallel writes + a small index file

Key idea

At scale, save **sharded**: every rank writes its own piece, in parallel.

Source: PyTorch documentation: torch.distributed.checkpoint (DCP).

Consistency: all or nothing



- **barrier**: no rank continues until every rank has arrived.
- **atomic**: happens completely or not at all — a rename is atomic.

Key idea

A checkpoint is either **complete and published**, or invisible.

Write, barrier, rename in code

```
tmp = f"ckpt/tmp-{step}"
save_shard(state, tmp, rank)   # every rank
dist.barrier()                 # all written
if rank == 0:
    os.replace(tmp, f"ckpt/step-{step}")
    point_latest_to(step)      # also atomic
dist.barrier()                 # wait for it
```

- Only rank 0 renames.
- A crash before the rename leaves only `tmp-...`: **ignored**.

Check

Restart loads whatever **latest** points to — always a complete checkpoint.

Source: Python documentation, os.replace; PyTorch documentation, torch.distributed.barrier.

How often to checkpoint

- **Too often**: time spent writing — δ every time.
- **Too rarely**: more work lost per failure — half an interval on average.

$$\text{best interval } \tau^* \approx \text{sqrt}(2\ \delta\ M)$$

δ: time one checkpoint blocks training; M: the job's MTBF

Result (Young, 1974)

$\delta = 1$ minute, M $\approx$ 3 hours: checkpoint about every **19 minutes**.

Source: J. W. Young, "A first order approximation to the optimum checkpoint interval," CACM 17(9), 1974.

Strong scaling

Same model, same **global batch**, same data and steps — more GPUs.

$$\text{speedup } S(n) = T(1) \;/\; T(n)$$

$$\text{efficiency } E(n) = S(n) \;/\; n, \;\; \text{ideally } 1$$

In our pipeline

T3 asks for **strong** scaling. Growing the batch with n changes the training problem itself.

Reproducible measurements

Control	How
Same problem	fixed seeds, global batch, data order, steps
Statistics	warm up; ≥ 3 repeats; report **median** and spread
Environment	record GPU, driver, and library versions

Watch out

Different n changes the all-reduce's **addition order**: expect agreement in statistics, not in bits.

Source: PyTorch documentation, "Reproducibility" notes.

Check yourself

Two minutes with a neighbour.

- Why must the checkpoint include the data position?
- $\delta = 30$ s, job MTBF $= 10$ hours: about how often should you checkpoint?
- What goes wrong if every rank writes directly into `ckpt/latest/`?

IN-CLASS BUILD

Diagnose a slow job

Work in pairs. Read the numbers; propose one fix.

Exercise: one step, eight ranks

Ranks	Compute	All-reduce kernel	Power
0–4, 6, 7	99–102 ms	47–50 ms	650 W
5	131 ms	18 ms	500 W

Every rank's step: **143 ms**

GIVEN

Data loading 4 ms

All-reduce alone 18 ms, 10 overlapped

Useful work 49 TFLOP per GPU

Peak 989 TFLOP/s per GPU

FIND

1. Straggler? Evidence? 2. Exposed communication? 3. MFU?
4. Energy per step? 5. One fix, and the new step time?

Solutions: diagnosis

- **Rank 5** computes for 131 ms vs $\approx$ 100 ms elsewhere.
- Its all-reduce is the **shortest** (18 ms); the others wait.
- It draws 500 W vs 650 W: likely **power-capped or throttling**.

$$143\ \text{ms} = 4\ (\text{data}) + 131\ (\text{rank } 5) + 8\ (\text{exposed})$$

Check

Rank 0's 49 ms all-reduce = **31 ms waiting** + 8 exposed + 10 overlapped.

Solutions: MFU, energy, and a fix

	Now	After the fix
Step time	143 ms	114 ms
MFU	35%	44%
Energy per step	722 J	$\approx$ 593 J

Fix: move off rank 5's GPU. The slowest remaining rank (102 ms) then sets the step.

Prediction

Predicted **1.25×** faster. Perfect overlap would save at most 8 ms: **fix the straggler first**, then re-profile.

Key takeaways

- Read the **compute row** first; only **exposed** communication lengthens a step.
- The **slowest rank** sets the step; a long all-reduce usually means **waiting** for it.
- "GPU utilization %" counts **time**, not work: report **MFU** and **energy** (power $\times$ time).
- Exact restart: model, optimizer, **RNG**, **data position** — saved by write, barrier, rename.
- Checkpoint about every **sqrt(2δM)**; scale with a fixed global batch, repeats, and medians.

Next time, and readings

When	What
This week	Capstone pairs are set
Week 14	Joint integration and review
Week 14	T3 due: DP vs DDP, TP/PP exercises, layout analysis, scaling study

READ

Young, "A first order approximation to the optimum checkpoint interval," CACM 1974 (two pages).

PyTorch documentation: *Distributed Checkpoint* (torch.distributed.checkpoint).